

Requêtes sur la Base de Données

— INFO-H303 - Bases de données —

31 mai 2026



Zalla Amine : 614603

Despau Bogdan : 612974

Rabeh Rayan : 576232

Cherkaoui Jawad : 576517

1. Les 10 utilisateurs ayant le plus de points

Algèbre relationnelle

Impossible. Il n'existe pas d'opérateur de tri en algèbre relationnelle standard, il est donc impossible d'ordonner les utilisateurs en fonction de leurs points. Il n'existe pas non plus d'opérateur de restriction permettant de limiter le résultat à 10 tuples.

Calcul tuple

Impossible. Même raisonnement que pour l'algèbre relationnelle : le calcul tuple retourne un ensemble de n-uplets sans notion d'ordre ni de limitation quantitative.

SQL

```
SELECT nomUtilisateur, nombrePoints, niveau
FROM Utilisateur
ORDER BY nombrePoints DESC, nomUtilisateur ASC
LIMIT 10;
```

2. Utilisateurs ayant publié des résumés dans au moins 3 cours différents

Algèbre relationnelle

On projette la table Resume sur (idUtilisateur, codeCours) et on en crée trois copies renommées afin d'exiger trois cours distincts :

$$\begin{aligned} R_1 &\leftarrow \pi_{\text{idUtil}, \text{code}}(\text{Resume}) \\ R_2 &\leftarrow \pi_{\text{idUtil}, \text{code}}(\text{Resume}) \\ R_3 &\leftarrow \pi_{\text{idUtil}, \text{code}}(\text{Resume}) \\ \text{Multi} &\leftarrow \pi_{\text{idUtil}}(R_1 \bowtie_{\text{idUtil}=\text{idUtil} \wedge c_1 \neq c_2} R_2 \bowtie_{\text{idUtil}=\text{idUtil} \wedge c_1 \neq c_3 \wedge c_2 \neq c_3} R_3) \\ \text{Résultat} &\leftarrow \pi_{\text{nomUtilisateur}}(\text{Utilisateur} \bowtie \text{Multi}) \end{aligned}$$

Calcul tuple

$$\{t \mid \text{Utilisateur}(t) \wedge \exists r_1, r_2, r_3 \in \text{Resume} (\\ r_1.\text{idUtilisateur} = t.\text{idUtilisateur} \wedge r_2.\text{idUtilisateur} = t.\text{idUtilisateur} \wedge \\ r_3.\text{idUtilisateur} = t.\text{idUtilisateur} \wedge \\ r_1.\text{codeCours} \neq r_2.\text{codeCours} \wedge \\ r_1.\text{codeCours} \neq r_3.\text{codeCours} \wedge \\ r_2.\text{codeCours} \neq r_3.\text{codeCours}) \}$$

SQL

```
SELECT u.nomUtilisateur,
       COUNT(DISTINCT r.codeCours) AS nb_cours, COUNT(r.idResume) AS
       nb_resumes
FROM Utilisateur u
JOIN Resume r ON r.idUtilisateur = u.idUtilisateur
GROUP BY u.idUtilisateur, u.nomUtilisateur
HAVING COUNT(DISTINCT r.codeCours) >= 3
ORDER BY nb_cours DESC, nb_resumes DESC, u.nomUtilisateur ASC;
```

3. Le cours ayant le plus grand nombre de résumés publiés

Algèbre relationnelle

Impossible. Il n'existe pas d'opérateur de comptage en algèbre relationnelle, ni d'opérateur permettant d'extraire la valeur maximale. Il est donc impossible de compter le nombre de résumés par cours et de sélectionner celui qui en possède le plus.

Calcul tuple

Impossible. Même raisonnement : le calcul tuple de premier ordre ne dispose d'aucun mécanisme pour agréger ou comparer des comptages.

SQL

```
SELECT c.codeCours, c.nomCours, c.credits, COUNT(r.idResume) AS
       nb_resumes
FROM Cours c
LEFT JOIN Resume r ON r.codeCours = c.codeCours
GROUP BY c.codeCours, c.nomCours, c.credits
ORDER BY nb_resumes DESC, c.nomCours ASC
LIMIT 1;
```

4. Les résumés les mieux notés (note moyenne maximale) pour chaque cours

Algèbre relationnelle

Impossible. Il n'existe pas d'opérateur permettant de calculer une moyenne arithmétique, ni d'opérateur pour extraire le maximum d'une valeur calculée. On ne peut ni obtenir la note moyenne d'un résumé, ni comparer ces moyennes pour sélectionner la plus élevée par cours.

Calcul tuple

Impossible. Même raisonnement : le calcul tuple de premier ordre ne permet pas d'exprimer des agrégations telles que la moyenne. L'expression est donc inexprimable.

SQL

```
-- Sous-requete 1 : moyenne des notes par resume
-- Sous-requete 2 : meilleure moyenne par cours

SELECT rr.codeCours, rr.nomCours, rr.credits, rr.titre, rr.avg_note
FROM (
    SELECT c.codeCours, c.nomCours, c.credits,
           r.idResume, r.titre,
           AVG(e.note) AS avg_note
    FROM Resume r
    JOIN Cours c ON c.codeCours = r.codeCours
    JOIN Evaluer e ON e.idResume = r.idResume
    GROUP BY c.codeCours, c.nomCours, c.credits, r.idResume, r.titre
) AS rr
JOIN (
    SELECT codeCours, MAX(avg_note) AS best_avg
    FROM (
        SELECT c.codeCours, r.idResume, AVG(e.note) AS avg_note
        FROM Resume r
        JOIN Cours c ON c.codeCours = r.codeCours
        JOIN Evaluer e ON e.idResume = r.idResume
        GROUP BY c.codeCours, r.idResume
    ) AS averages
    GROUP BY codeCours
) AS best_per_course
ON best_per_course.codeCours = rr.codeCours
AND best_per_course.best_avg = rr.avg_note
ORDER BY rr.nomCours ASC, rr.titre ASC;
```

5. Les utilisateurs n'ayant jamais publié de résumé

Algèbre relationnelle

On utilise l'opérateur de différence ensembliste pour retrancher les utilisateurs ayant au moins un résumé :

$$\begin{aligned}\text{TousUtilisateurs} &\leftarrow \pi_{\text{nomUtilisateur}, \text{idUtilisateur}}(\text{Utilisateur}) \\ \text{Auteurs} &\leftarrow \pi_{\text{nomUtilisateur}, \text{idUtilisateur}}(\text{Utilisateur} \bowtie \text{Resume}) \\ \text{Résultat} &\leftarrow \pi_{\text{nomUtilisateur}}(\text{TousUtilisateurs} - \text{Auteurs})\end{aligned}$$

Calcul tuple

$$\{t \mid \text{Utilisateur}(t) \wedge \neg \exists r \in \text{Resume}(r.\text{idUtilisateur} = t.\text{idUtilisateur})\}$$

SQL

```
SELECT u.nomUtilisateur, u.email, u.nombrePoints
FROM Utilisateur u
LEFT JOIN Resume r ON r.idUtilisateur = u.idUtilisateur
WHERE r.idResume IS NULL
ORDER BY u.nomUtilisateur ASC;
```

6. L'objet cosmétique le plus acheté

Algèbre relationnelle

Impossible. Il n'existe pas d'opérateur de comptage ni d'opérateur permettant d'extraire le maximum. On ne peut donc pas compter le nombre d'achats par objet ni sélectionner le plus acheté.

Calcul tuple

Impossible. Même raisonnement : le calcul tuple de premier ordre ne dispose d'aucune primitive d'agrégation ni de comparaison de comptages.

SQL

```
SELECT oc.idObjet, oc.nomObjet, oc.description, oc.prixPoints,
       COUNT(p.idobjet) AS purchase_count
FROM ObjetCosmetique oc
LEFT JOIN Possede p ON p.idObjet = oc.idObjet
GROUP BY oc.idObjet, oc.nomObjet, oc.description, oc.prixPoints
ORDER BY purchase_count DESC, oc.nomObjet ASC
LIMIT 1;
```

7. Utilisateurs ayant dépensé plus de points qu'ils n'en possèdent

Algèbre relationnelle

Impossible. Il n'existe pas d'opérateur de somme arithmétique en algèbre relationnelle. On ne peut donc pas cumuler les dépenses d'un utilisateur pour les comparer à son solde actuel.

Calcul tuple

Impossible. Non exprimable en calcul tuple de premier ordre cela nécessite de réaliser une somme sur une colonne numérique, ce qui dépasse les capacités de ce formalisme.

SQL

```
SELECT u.nomUtilisateur,
       u.nombrePoints,
       COALESCE(SUM(tp.montantPoints), 0) AS total_spent,
       COALESCE(SUM(tp.montantPoints), 0) - u.nombrePoints AS
         excess_spent
FROM Utilisateur u
LEFT JOIN TransactionPoints tp
  ON tp.idUtilisateur = u.idUtilisateur
  AND tp.natureTransaction = 'depense'
GROUP BY u.idUtilisateur, u.nomUtilisateur, u.nombrePoints
HAVING COALESCE(SUM(tp.montantPoints), 0) > u.nombrePoints
ORDER BY excess_spent DESC, total_spent DESC, u.nomUtilisateur ASC;
```

8. Le nombre moyen de résumés publiés par utilisateur

Algèbre relationnelle

Impossible. L'algèbre relationnelle ne dispose pas d'opérateur permettant de compter globalement les éléments d'une table, ni d'effectuer une division arithmétique pour obtenir une moyenne. Cette requête est donc inexprimable.

Calcul tuple

Impossible. Inexprimable. Le calcul tuple retourne un ensemble de n-uplets et non une valeur scalaire issue d'opérations mathématiques globales telles qu'une moyenne.

SQL

```
-- Moyenne calculée sur tous les utilisateurs
-- (y compris ceux sans resume).

SELECT AVG(user_resume_count) AS moyenne_resumes
FROM (
  SELECT u.idUtilisateur, COUNT(r.idResume) AS user_resume_count
  FROM Utilisateur u
  LEFT JOIN Resume r ON r.idUtilisateur = u.idUtilisateur
  GROUP BY u.idUtilisateur
) AS per_user;
```